numberAI: Building an Image Classifier from Scratch

Project Documentation

May 15, 2026

Abstract

This document outlines the development, architecture, and findings of the **numberAI** project. The objective of this project is to recreate machine learning functionality from scratch, without relying on pre-trained models or established neural network structures. By utilizing simple matrix multiplication and a trial-and-error training approach, the project aims to categorize number images. Additionally, it details the planned second phase involving quantization and distillation for deployment on resource-constrained embedded systems such as Arduino.

Contents

1 Introduction **3**
- 1.1 Project Motivation . . . 3
- 1.2 Goals and Phases . . . 3

2 Phase 1: Architecture Iterations **3**
- 2.1 Version 0: Initial Attempt . . . 3
  - 2.1.1 Architecture and Methodology . . . 3
  - 2.1.2 Results and Failures . . . 4
  - 2.1.3 Points to Solve in Next Iteration . . . 4
- 2.2 Version 1.0: Non-Linearity and Optimization . . . 4
  - 2.2.1 Architecture Upgrades . . . 4
  - 2.2.2 Methodology Upgrades . . . 5
  - 2.2.3 Version 1.0 Training Results . . . 5
  - 2.2.4 Points to Solve in Next Iteration . . . 6
- 2.3 Version 1.1: Vectorized Blame Assignment (Backpropagation) . . . 7
  - 2.3.1 Methodology Upgrades . . . 7
  - 2.3.2 Version 1.1 Training Results . . . 7
  - 2.3.3 Points to Solve in Next Iteration . . . 8
- 2.4 Version 1.2: Improved Dataset Generalization . . . 9
  - 2.4.1 Methodology and Results . . . 9
  - 2.4.2 Points to Solve in Next Iteration . . . 10
- 2.5 Version 2.0: Introduction of Convolution . . . 10
  - 2.5.1 Architecture Upgrades . . . 10
  - 2.5.2 Version 2.0 Training Results . . . 10
  - 2.5.3 Points to Solve in Next Iteration . . . 10
- 2.6 Version 2.1: PyTorch Optimization . . . 10

2.6.1 Architecture and Methodology Upgrades 11
2.6.2 Version 2.1 Training Results 11
2.6.3 Points to Solve in Next Iteration 12

3 Phase 2: Embedded Systems Optimization 12
3.1 Quantization 12
3.2 Distillation 12

4 Discoveries and Challenges 13
4.1 Model Selection via Relativistic Efficiency Analysis 13
4.1.1 The "Escape Velocity" Constraint 13
4.2 Matrix Size Optimization 14
4.3 Trial and Error Efficiency 14

5 Conclusion 14

1 Introduction

1.1 Project Motivation

Modern machine learning often relies on complex, pre-defined architectures and pre-trained weights. The motivation behind numberAI is to deconstruct this paradigm and build an image classifier from the absolute basics to fully understand the underlying mechanics of neural networks.

1.2 Goals and Phases

The project is divided into two main phases:

- **Phase 1: Foundational Discovery.** To categorize number images using pure matrix multiplication. This phase avoids pre-defined structures, instead relying on trial and error to iteratively discover the optimal matrix size and network topology.
- **Phase 2: Embedded Systems Deployment.** To run the developed classification model on embedded systems with limited resources (e.g., Arduino). This requires applying model compression techniques such as *quantization* and *distillation* to reduce memory footprint and computational load.

2 Phase 1: Architecture Iterations

As the project employs a trial-and-error discovery method, the model architecture and training methodology evolved chronologically through multiple versions. This section documents the progression of the model, detailing the problems encountered in each version and the subsequent upgrades.

2.1 Version 0: Initial Attempt

The very first attempt (**Version 0**) served as a baseline to test the pure matrix multiplication concept without any additional complexity.

2.1.1 Architecture and Methodology

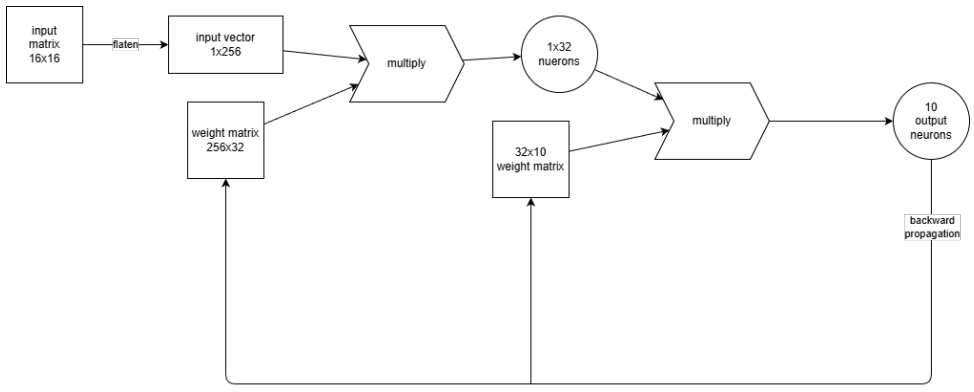

Figure 1: Version 0 Architecture

- **Linear Topology**: The network consisted of two weight matrices ($H1$ and $H2$) with no activation functions between them.
- **I/O Inefficiency**: The training loop read and wrote the matrices to the disk at every single iteration.

2.1.2 Results and Failures

Version 0 was ultimately unsuccessful due to several critical flaws:

- **Linearity Collapse**: Without a non-linear activation function, the two-layer network was mathematically pointless. Sequential linear matrix multiplications can be collapsed into a single matrix, meaning no deep representations were being learned.
- **Performance Bottleneck**: Reading and writing matrices to the disk at every step made the training process extremely slow.
- **Gradient Explosion**: The training never finished because the loss would eventually hit NaN due to gradient explosions. Consequently, no usable model was produced for testing.

2.1.3 Points to Solve in Next Iteration

- Introduce a non-linear activation function (e.g., ReLU) to prevent linearity collapse and enable deep feature learning.
- Optimize the matrix disk read/write methodology to eliminate the severe I/O performance bottleneck during training.

2.2 Version 1.0: Non-Linearity and Optimization

To address the fatal flaws of Version 0, several major upgrades were introduced in **Version 1.0**.

2.2.1 Architecture Upgrades

The structure for Version 1.0 introduced non-linearity:

- **Input Layer**: Flattened representation of the input image.
- **Hidden Layers**: Two primary weight matrices ($H1$ and $H2$). Dimensions were iteratively tested (e.g., $256 \times 32 \rightarrow 32 \times 10$, and later $4096 \times 128 \rightarrow 128 \times 10$).
- **Activation Function**: ReLU (Rectified Linear Unit) was introduced between the hidden layers to prevent the linearity collapse seen in Version 0.
- **Output Layer**: Vector of size 10×1, representing the classification scores for digits 0-9.

2.2.2 Methodology Upgrades

- **Optimized I/O**: The matrix reading and writing methods were optimized to keep weights in memory during the training loop, massively speeding up training.
- **Numerical Gradient Training**: The loss function calculates the sum of Euclidean distances between the actual output vector (**A**) and expected output (**EA**):

$$\mathbf{Loss} = \sum_{k=0}^{9} \sqrt{\sum_{n=0}^{9} \left(A_{k,n} - EA_{k,n}\right)^2} \tag{1}$$

The weight was then updated using a numerical gradient approximation (trial-and-error):

$$H_{m,n} := H_{m,n} - \left(\frac{\mathbf{Loss}_{o+\text{change}} - \mathbf{Loss}_{o}}{\text{change}} \right) \cdot \alpha \tag{2}$$

2.2.3 Version 1.0 Training Results

After training the Version 1.0 architecture, a measurable drop in Shannon entropy for the weight matrices was observed:

- **Pre-training Entropy**: $H1 \approx 6.63$, $H2 \approx 6.45$
- **Post-training Entropy**: $H1 \approx 3.05$, $H2 \approx 3.23$

This lower entropy indicates that the trial-and-error process successfully converged on structured patterns within the dataset. A visualization of the training loss can be seen below:

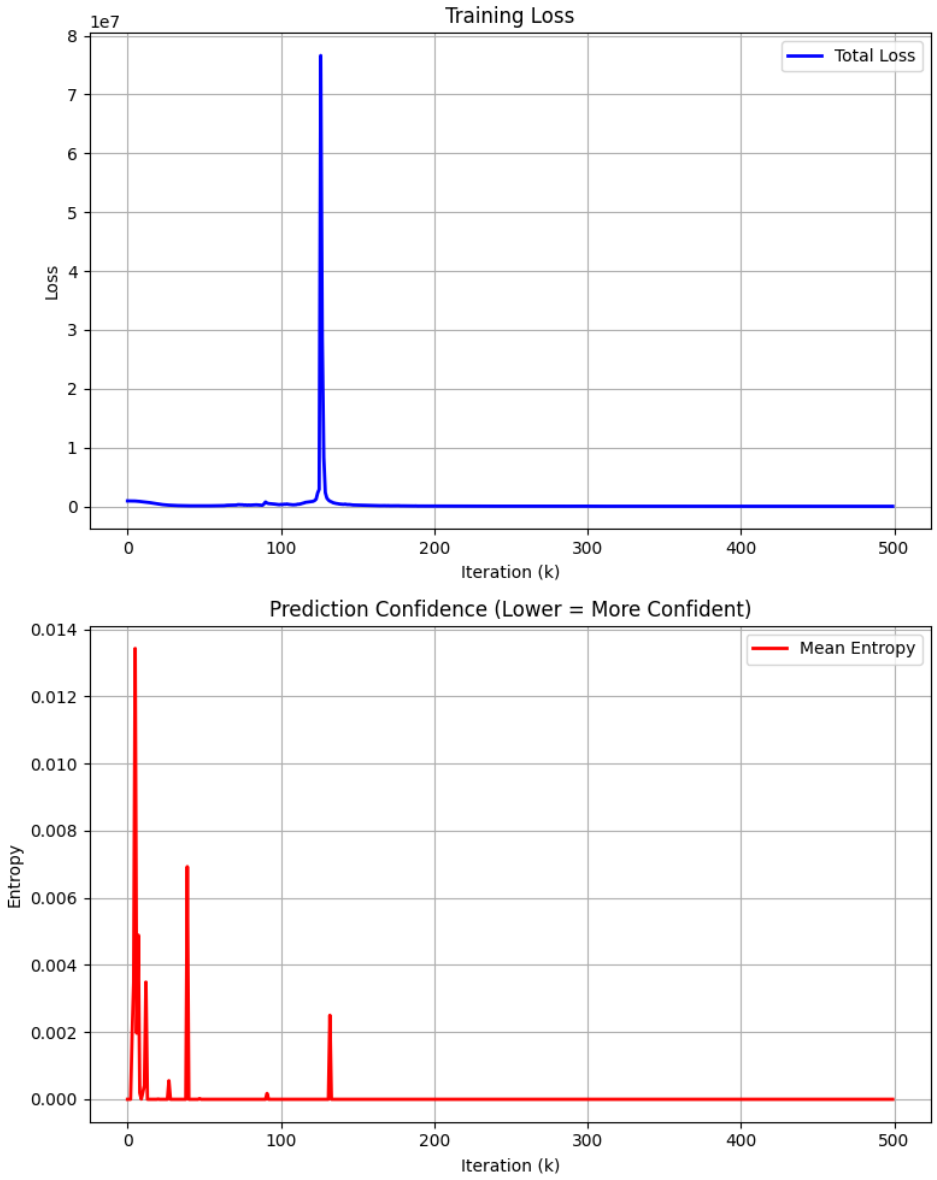

Figure 2: Version 1.0 Training Loss Progression

Overfitting Discovery: While the training process converged, evaluating the model revealed a severe case of overfitting. The model successfully categorized all images in the training dataset, achieving an accuracy of 100.00% (100/100). However, when exposed to new, unseen number images from the verification dataset, it completely failed, yielding an accuracy of 0.00% (0/10). This indicates that the Version 1.0 architecture effectively memorized the specific pixels of the training data but failed to generalize the underlying visual features of the numbers.

2.2.4 Points to Solve in Next Iteration

- Address the severe **overfitting** to ensure the model generalizes to unseen data instead of merely memorizing the training dataset.
- Improve **training efficiency**. The numerical gradient approach is far too slow (taking upwards of 15 minutes) for a model of this size.

- **Loss Function Singularity**: The Euclidean distance loss function uses a square root. Its derivative places the square root in the denominator, meaning as the loss approaches zero, it causes a divide-by-zero error leading to NaN gradient explosions. A loss function without this singularity (such as Mean Squared Error) is required.

2.3 Version 1.1: Vectorized Blame Assignment (Backpropagation)

To address the severe overfitting and slow training times observed in Version 1.0, the training methodology was significantly overhauled in **Version 1.1**. The numerical gradient approach (trial-and-error finite difference) was replaced with a heavily optimized, mathematically grounded blame assignment method.

2.3.1 Methodology Upgrades

The adjustment logic was optimized using vectorized blame assignment (backpropagation):

- **H2 Adjustment**: The error vector is multiplied by the transposed middle neuron vector (from $H1$) to compute the gradient for all parameters of $H2$ simultaneously.
- **H1 Adjustment**: The error vector is multiplied by the transposed $H2$ matrix, passed through the derivative of the ReLU function, and then multiplied by the transposed input vector (I). This allows the simultaneous adjustment of all $H1$ parameters.

This vectorized approach eliminates the need to perturb weights one-by-one, replacing millions of individual forward passes with a single efficient backward pass.

2.3.2 Version 1.1 Training Results

The implementation of vectorized blame assignment yielded two major breakthroughs:

1. **Blazing Fast Training**: The training time was drastically reduced to approximately 30.6 seconds (down from over 15 minutes in Version 1.0), completing 500 iterations over the entire dataset with ease.
2. **Generalization over Memorization**: The updated weight adjustment logic successfully prevented the model from merely memorizing pixels. Evaluation metrics showed:
    - **Training Dataset Accuracy**: 100.00% (100/100)
    - **Verification Dataset Accuracy**: 60.00% (6/10)

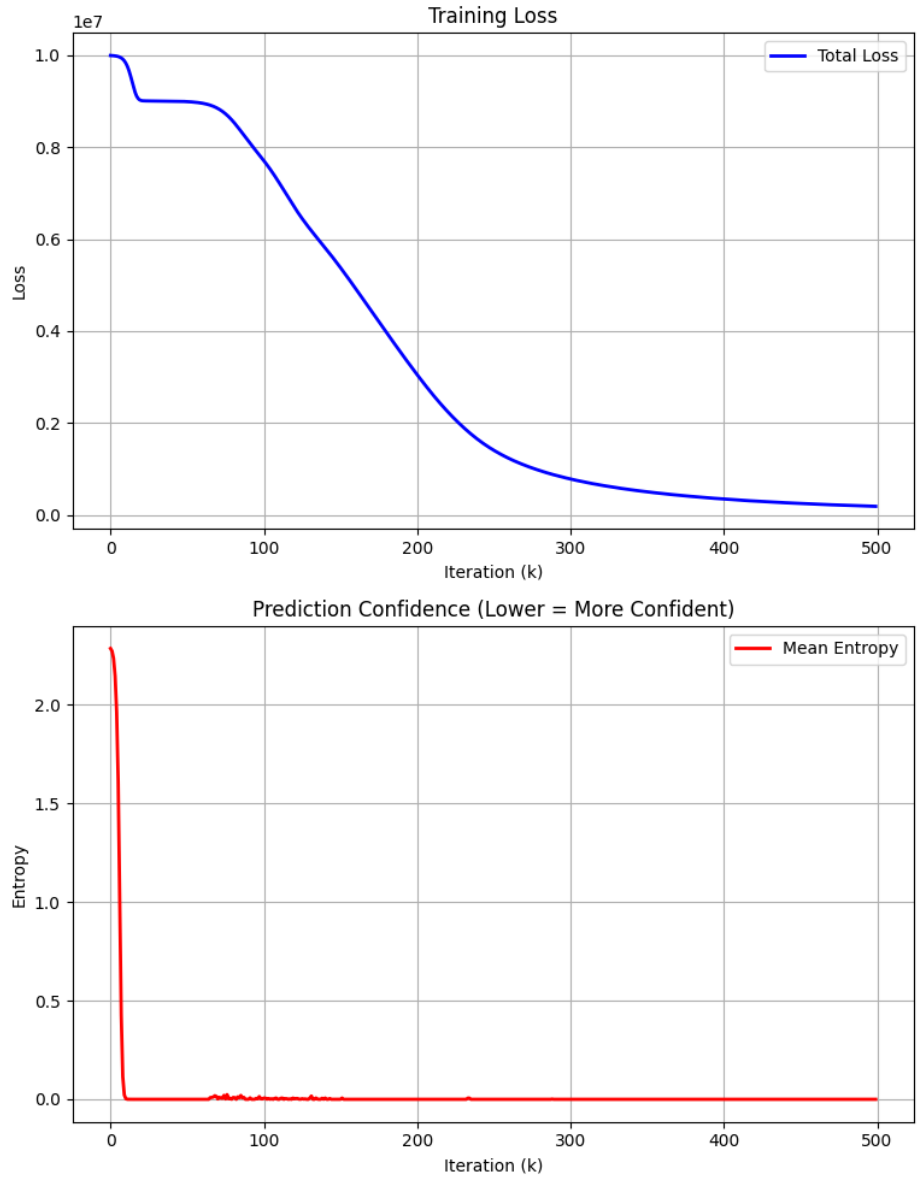

Figure 3: Version 1.1 Training Loss Progression

Learning Curve Analysis & The Memorization Phase:
As seen in Figure 3, the learning curve exhibits unusual behavior. The model initially learns rapidly, but quickly hits a plateau where the blame assignment algorithm no longer encounters novel information, even though the loss remains relatively high. Because the training dataset is severely limited (only 100 images), the model exhausts the generalizable patterns and enters a *memorization phase*. Instead of learning the underlying features of the numbers, it begins to memorize the specific 100 images to force the loss lower. This perfectly explains why the training accuracy reaches 100% while the verification accuracy stagnates at 60%.

2.3.3 Points to Solve in Next Iteration

- **Dataset Starvation**: The core cause of the memorization plateau is the extremely small dataset. The model needs a vastly larger and more diverse dataset to prevent

it from memorizing and force it to continuously learn generalizable features.

- **Dynamic Stopping Condition**: Currently, training terminates after a hardcoded 500 iterations without evaluating the model's actual convergence state. The next version needs a dynamic stopping mechanism based on metrics such as Shannon Entropy, the slope of the loss, or the raw loss value itself.
- **Adaptive Learning Rate**: The learning rate parameters were chosen via arbitrary guessing. Incorrect learning rates caused several initial training attempts of Version 1.1 to fail. A systematic or adaptive method for determining the optimal learning rate is necessary for stable and reliable training.

2.4 Version 1.2: Improved Dataset Generalization

To address the generalization issues identified in Version 1.1, the training dataset was fundamentally upgraded in **Version 1.2**. The custom, small-scale dataset was replaced with the standardized MNIST dataset.

2.4.1 Methodology and Results

By training the model on the much larger and highly varied MNIST dataset, the model was exposed to a far wider array of handwriting styles. This significantly improved the model's ability to extract general features rather than memorizing specific pixels.

- **Generalization Success**: The model achieved a highly promising **93.93% accuracy** on the verification set, resolving the core overfitting issue from earlier versions.

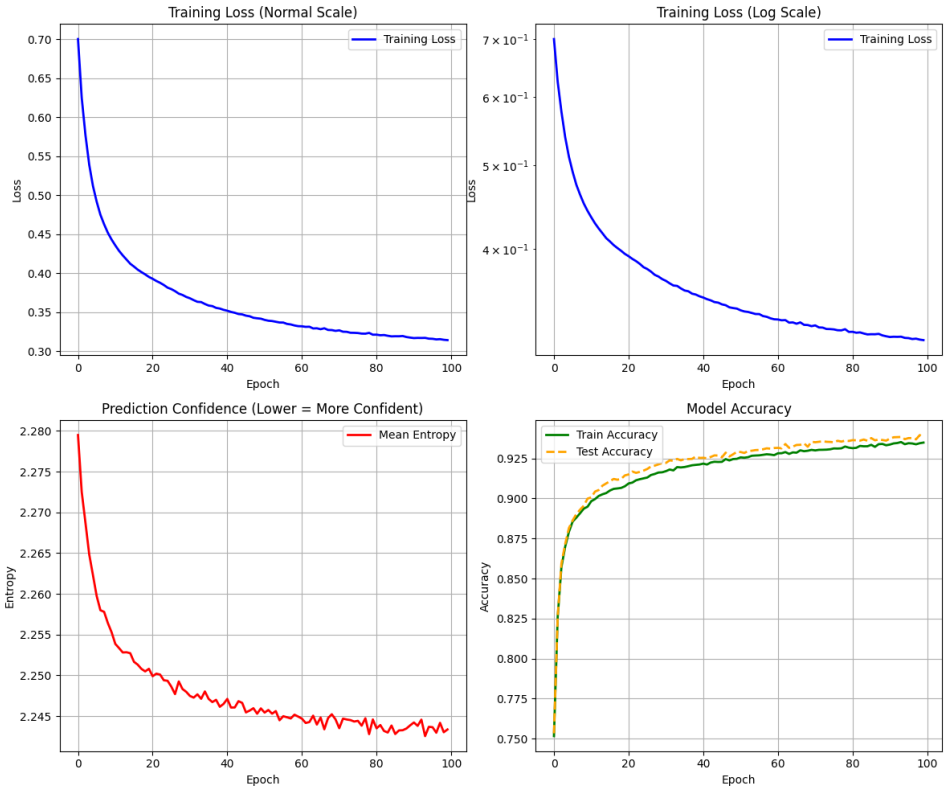

Figure 4: Version 1.2 Training Loss Progression

2.4.2 Points to Solve in Next Iteration

- **Shifted Number Problem**: While the model performs well on centered digits, its reliance purely on fully connected dense layers means it is highly sensitive to spatial translation. If a number is shifted off-center, the model struggles to categorize it accurately. A mechanism to introduce translation invariance is required.

2.5 Version 2.0: Introduction of Convolution

To solve the shifted number problem and reduce the overall model size, **Version 2.0** introduced a custom convolution operation before the fully connected layers.

2.5.1 Architecture Upgrades

- **Convolutional Kernel**: A single 3×3 learnable kernel (K) was introduced. The input image (28×28) is convolved with this kernel to produce a feature map.
- **Max Pooling**: A 2×2 max pooling layer was implemented to downsample the feature map, providing the desired translation invariance and reducing the dimensionality of the data before it enters the dense layers ($H1$ and $H2$).

2.5.2 Version 2.0 Training Results

The introduction of the convolutional kernel successfully enabled the model to handle shifted numbers, and it maintained an accuracy comparable to Version 1.2. However, this architecture introduced new bottlenecks:

- **Computational Overhead**: The custom Python implementation of the convolution and pooling operations significantly increased the training time compared to the pure matrix multiplication of Version 1.2.
- **Feature Limitation**: Using only a single convolution kernel restricted the model to focusing on just one specific spatial characteristic (e.g., a single edge type or texture), preventing it from learning rich, multi-dimensional features.

2.5.3 Points to Solve in Next Iteration

- **Optimize Training Speed**: The custom numpy convolution loops are too slow. A highly optimized framework backend is necessary for practical training times.
- **Expand Feature Extraction**: The architecture must support multiple convolution kernels simultaneously to capture diverse features (e.g., horizontal edges, vertical edges, curves).

2.6 Version 2.1: PyTorch Optimization

To address the computational bottlenecks of the custom numpy convolution implementation in Version 2.0, **Version 2.1** transitions the model to leverage the PyTorch framework, utilizing GPU acceleration (CUDA).

2.6.1 Architecture and Methodology Upgrades

By adopting PyTorch, the network successfully vectorizes its operations, replacing the slow Python loops from Version 2.0 with optimized tensor operations while preserving the exact same hand-written backpropagation math. This massive computational speedup allowed the architecture to be scaled:

- **Multiple Kernels**: The model was successfully upgraded from a single kernel to using **8 convolution kernels**. This enables the network to simultaneously extract multiple distinct spatial features (edges, curves, etc.) from the input images.
- **Matrix Resizing**: The hidden layer dimensions were adjusted to accommodate the multi-channel output from the convolutional and max pooling layers, resulting in $H1$ expanding to 1352×64, and $H2$ adjusting to 64×10.

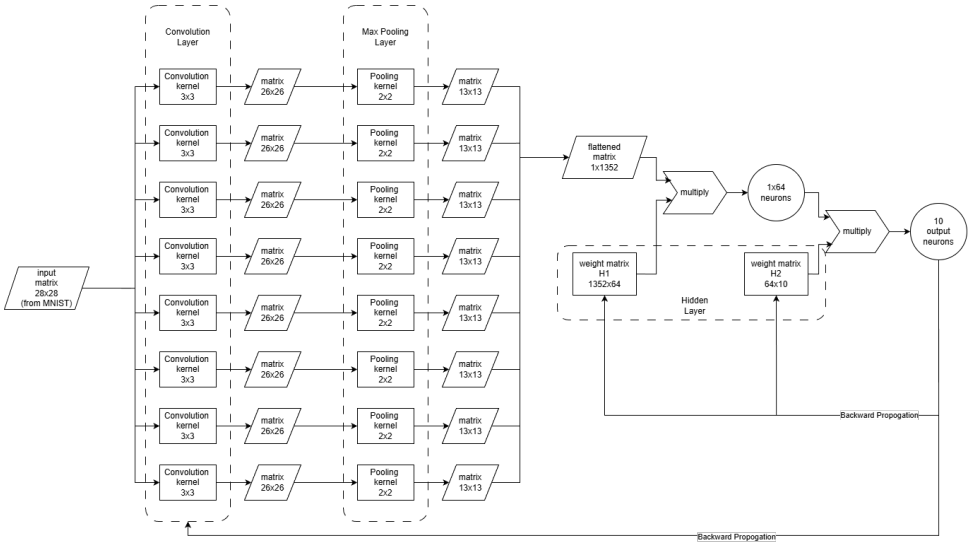

Figure 5: Version 2.1 Architecture

2.6.2 Version 2.1 Training Results

The framework upgrade yielded incredible performance improvements:

- **Training Speed**: A baseline test using 1 kernel and 32 neurons took only **57 seconds** to train (a massive reduction from the $\approx$ 1150 seconds required by the numpy implementation). Because of this efficiency, scaling the network up to the full 8 kernels and 64 neurons still resulted in a highly practical training time of just **324 seconds**.
- **MNIST Accuracy**: With the expanded 8-kernel architecture, the model achieved a highly impressive **94.68%** accuracy on the standardized MNIST dataset.

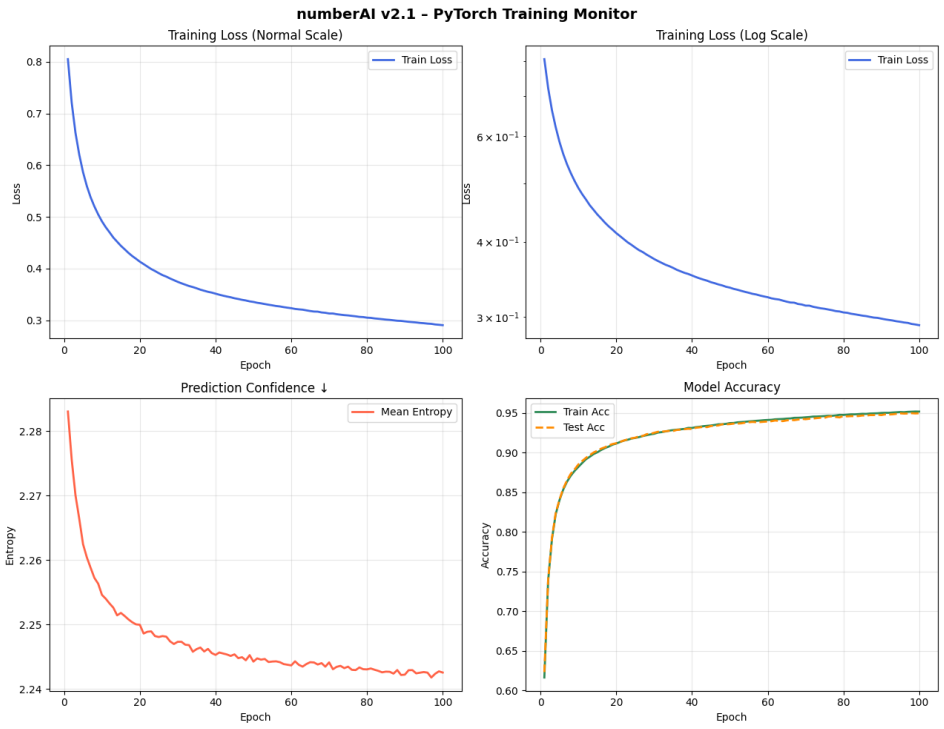

Figure 6: Version 2.1 Training Loss Progression (8 Kernels)

2.6.3 Points to Solve in Next Iteration

- **Further Accuracy Tuning**: While 94.68% is an excellent baseline for a hand-written CNN, standard architectures often achieve ¿98% on MNIST. Future versions can explore adding a second convolutional layer (deepening the network) or introducing dropout to push the accuracy even higher.

3 Phase 2: Embedded Systems Optimization

To deploy this model on an Arduino, the network must be significantly compressed.

3.1 Quantization

[Placeholder: Describe the process of converting floating-point matrix weights to lower precision formats (e.g., 8-bit integers) and the resulting impact on accuracy.]

3.2 Distillation

[Placeholder: Describe how a smaller, more efficient model (student) is trained to replicate the behavior of the larger, successful trial-and-error model (teacher).]

4 Discoveries and Challenges

4.1 Model Selection via Relativistic Efficiency Analysis

Following the implementation of Version 2.1, a critical architectural crossroad was reached: choosing between a resource-intensive model with slightly higher accuracy (8 kernels, 324s training) and a cheaper model with faster training time but slightly lower accuracy (1 kernel, 57s training).

In machine learning, achieving an accuracy increase from 94% to 96% requires exponentially more computational effort than jumping from 10% to 12%. This diminishing return perfectly mirrors the principles of Special Relativity, where accelerating an object closer to the speed of light requires increasingly more energy. To formally evaluate model efficiency, an analytical method was developed adapting the relativistic kinetic energy equation:

$$E = \frac{mc^2}{\sqrt{1 - \frac{v^2}{c^2}}} - mc^2 \tag{3}$$

Within the context of neural network optimization, the variables are redefined as follows:

- **Velocity (v)** represents the model's accuracy. Because an untrained model initialized with random weights naturally achieves a baseline 10% accuracy on a 10-digit dataset, the raw accuracy is linearly normalized:

$$v_{acc} = \frac{\text{Accuracy} - 0.1}{0.9} \tag{4}$$

  Here, a perfect 100% accuracy translates to $v_{acc} = 1.0$, conceptually equivalent to the speed of light (c).

- **Mass (m)** represents the baseline structural complexity or "action" of the number recognition task.

- **Energy (E)** represents the absolute computational cost of the model. In practical terms, this is quantified as: Parameter Size $\times$ Run Time.

Because the exact value of the conceptual mass (m) is unknown, absolute energy calculations for a single model are impossible. However, when comparing two distinct models, relative scaling can be employed. By calculating the theoretical relativistic energy ratio required to achieve Model B's accuracy compared to Model A's accuracy, and checking that against the *actual* computational cost ratio, true efficiency can be determined. For instance, if Model B requires 10x more theoretical energy (due to higher accuracy) but only incurs 2x more actual computational cost, Model B is objectively the more energy-efficient architecture.

4.1.1 The "Escape Velocity" Constraint

A mathematical flaw in pure efficiency analysis is that an extremely cheap, low-accuracy model might appear mathematically optimal but remain practically useless. Drawing inspiration from rocket engineering, an "escape velocity" constraint was established. The absolute minimum baseline for a model to be considered viable is an accuracy of **90%**.

Any model failing to reach this escape velocity is automatically disqualified from selection, regardless of its theoretical efficiency.

This relativistic analysis framework allows for highly objective, mathematically sound decisions when balancing model architecture tradeoffs, ensuring that added computational costs are truly justified by the resulting accuracy gains.

4.2 Matrix Size Optimization

[Placeholder: Discuss which matrix dimensions yielded the best balance between accuracy and computational complexity.]

4.3 Trial and Error Efficiency

[Placeholder: Discuss the challenges faced when training without traditional gradient descent. What heuristics were discovered to speed up convergence?]

5 Conclusion

[Placeholder: Summarize the final outcomes of the project, whether the goals were fully met, and potential future improvements.]